



Lab: Run a Simple AI Workload Using a Hosted GPU (Google Colab)



Lab Objective

Run a simple deep learning model (image classifier) using a GPU hosted in Google Colab. You'll:

- Enable GPU runtime
- Load a dataset
- Train a model using TensorFlow or PyTorch
- Observe the speedup vs CPU
- Understand GPU memory usage



Requirements

- Google account
- Browser access to <https://colab.research.google.com>



Step-by-Step Instructions

Step 1: Open Colab and Enable GPU

1. Visit: <https://colab.research.google.com>
2. Click "**File > New Notebook**"
3. Go to **Runtime > Change runtime type**
4. Under **Hardware accelerator**, select **GPU**
5. Click "**Save**"
6. In a new code cell, type:

```
!nvidia-smi
```

7. Run the cell by pressing `Shift + Enter`



You should see info about a Tesla T4 or similar GPU.

Step 2: Install and Import Libraries (if needed)

Colab often comes with everything preinstalled. But if needed:

```
# Uncomment if needed
# !pip install tensorflow
# or
# !pip install torch torchvision
```

Then, import required libraries:

```
import tensorflow as tf
from tensorflow.keras import layers, models
import numpy as np
import matplotlib.pyplot as plt
```

✅ This confirms TensorFlow is installed and GPU-ready.

Step 3: Load a Simple Dataset (e.g., MNIST)

```
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

# Normalize and reshape
x_train = x_train / 255.0
x_test = x_test / 255.0

x_train = x_train[..., np.newaxis]
x_test = x_test[..., np.newaxis]
```

✅ MNIST is a small dataset of 28x28 grayscale digits—perfect for quick GPU testing.

Step 4: Build a Simple CNN Model

```
model = models.Sequential([
    layers.Conv2D(32, kernel_size=(3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D(pool_size=(2, 2)),
    layers.Conv2D(64, kernel_size=(3, 3), activation='relu'),
    layers.MaxPooling2D(pool_size=(2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])
```

✅ This model has ~1M parameters—small enough to train quickly, but enough to engage the GPU.

Step 5: Compile and Train the Model on GPU

```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Train the model
model.fit(x_train, y_train, epochs=3, batch_size=64, validation_split=0.1)
```

🕒 You'll see training progress for each epoch. Notice how fast each epoch completes—this is due to GPU acceleration.

Step 6: Evaluate the Model

```
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"Test accuracy: {test_acc:.2f}")
```

✅ This confirms the model was trained correctly and can generalize to unseen data.

Step 7: Check GPU Utilization

To see GPU memory usage during training:

```
!nvidia-smi
```

🕒 Try running it during and after training.

Optional: Compare with CPU Runtime

To compare with CPU-only training:

1. Go to **Runtime > Change runtime type**
2. Set **Hardware accelerator = None**
3. Re-run the notebook from scratch

🕒 Training will now be **much slower**—a great way to see GPU impact firsthand.

What You Learned

- How to launch a GPU-enabled Colab notebook
 - How to train a neural network using TensorFlow
 - How GPU acceleration improves deep learning speed
 - Basic visibility into GPU utilization
-